Stage 2 Implementation Report

AI-powered Study Planner | EECS 3311 | Rafat Kabir
Progress draft: F01–F03, November 1, 2026.

1. Overview and status

Students can create courses, upload materials, and generate document summaries through a Vaadin GUI or Java CLI. Spring Boot stores courses, materials, and summaries in H2. F03 uses local Ollama; the HTTP adapter also supports hosted OpenAI-compatible providers.

F01–F03 are implemented. Editing/deletion and F04–F10 remain pending. Authentication supports one locally configured account and does not count as a project feature.

2. Feature-to-implementation mapping

F01: Course Creation. Deterministic. References: Stage 1 Task 1.2 F01, UC-01, SD01, and Task 4 F01. Students enter a name, description, start date, and end date. The system validates the input, saves an owned course, and displays it on the dashboard. Video: pending.

Part	Files, responsibilities, and key methods
GUI	`frontend/.../views/MainView.java`: `showSignIn`, `showDashboard`, `openCourseForm`, `refreshCourses`, and `renderCourses` handle sign-in, forms, errors, and course cards. `themes/study-planner/styles.css` styles the UI.
API client	`frontend/.../course/CourseApi.java`: `createCourse`/`viewCourses` call the backend. `CourseApiFactory.connect` creates a client for each signed-in view. Frontend `Course` and `CreateCourseRequest` records carry API data.
REST API	`backend/.../course/CourseController.java`: `createCourse`, `viewCourses`, and `getCourse` expose POST/GET `/api/courses` and GET `/api/courses/{id}`.
Application and domain	`CourseService` coordinates creation and retrieval. `User.createCourse/viewCourses` create and retrieve owned courses. Backend `Course` validates domain data; `CreateCourseRequest` validates API fields.
Storage	`CourseRepository` defines storage operations. `JdbcCourseRepository.save/findByOwner/findByIdAndOwner` use SQL. `schema.sql` defines the table. H2 files persist under `backend/data/`.

Authentication and errors	`SecurityConfiguration` verifies credentials; `CsrfController.token` supports protected writes. `ApiExceptionHandler` handles invalid requests and storage failures. The authenticated student determines course ownership.
CLI	`cli/StudyPlannerCli.java`: `main/run` provide `create`, `list`, and `help` using the same authenticated API.

Flow: form or CLI → CourseController → CourseService → User.createCourse → CourseRepository → H2. The GUI then reloads the saved course list. Failed submissions show errors and preserve form input.

All four fields are required. Names allow 120 characters; descriptions allow 2000. Text is trimmed, and end dates must be on or after start dates. Same-day courses and duplicate names are allowed. Course queries are restricted to their owner.

F02: Upload Learning Materials. References: Stage 1 Task 1.2 F02, UC-02, SD01, and Task 4 F02. Open an owned course, select files, and view the saved materials and extracted text. Upload/validation/text extraction are deterministic; the planned AI processing belongs to later features. Video: pending.

Part	Implementation and key methods
GUI / client	`MainView.openCourse` opens `CourseMaterialsPanel`; upload, `refreshMaterials`, and `viewText` show results and errors. Each file offers View text and Download original. `CourseApi.uploadMaterial/listMaterials/materialContent/downloadMaterial` call the authenticated API.
API / service	`CourseController.uploadMaterial/listMaterials/materialContent/downloadMaterial` delegate to `MaterialService`, which verifies course ownership before access.
Domain / parsing	`Course.addMaterial` creates `LearningMaterial`; `getContent` exposes extracted text. `MaterialParser.parse` checks filenames and parses PDF, DOCX, or UTF-8 TXT using PDFBox, POI, or UTF-8 decoding.
Storage / CLI	`MaterialRepository` and `JdbcMaterialRepository` save metadata, original BLOB, and extracted text together. `StudyPlannerCli` adds `upload`, `materials`, and `content`.

Rules: up to 10 files, 10 MiB each; unsupported, empty, corrupt, encrypted, or text-free files are rejected. No OCR is provided. Text is limited to 1,000,000 characters, PDF to 500 pages, and expanded DOCX to 20 MiB/1000 entries. API batches roll back on any failure; GUI files upload independently. Originals and text survive restarts.

F03: Document Summarization. AI-based. References: Stage 1 F03, UC-03, SD02 summary branch, and Task 4 F03. Select Summarize document; the agent summarizes source sections, synthesizes them when needed, resolves selected source-excerpt IDs to verified quotations, and saves the result. View saved summary reuses it; regeneration explicitly requests a replacement. CLI: `summarize`, `summary`, `regenerate-summary`. Video pending.

Part	Implementation
GUI / API	`frontend/.../views/SummaryActions` runs background requests; `CourseApi.summarize/viewSummary` call `backend/.../resource/SummaryController`. Progress and safe errors remain visible while inference runs.
Agent / resource	`Course.generateStudyResource` calls `StudyAssistantFacade.summarize`. `ContentAgent.summarize` plans sections and validates responses from `AIEngine.generate`. `StudyResourceFactory.createResource` constructs `Summary`, extending `StudyResource`; `view` returns its text.
Storage	`SummaryService` verifies ownership, guards concurrent generation, and checks the source hash. `SummaryRepository/JdbcSummaryRepository` save the latest summary, evidence, model, and usage. Failed regeneration preserves the previous result.

F03 accepts 40–32,000 source characters. Generation has schema-constrained output, bounded calls/time, and one repair attempt for invalid results. Missing setup, provider failures, insufficient text, invalid output, and timeouts return errors without saving. Quotations must match the source; factual accuracy still needs human review.

Remaining Stage 1 features are **not implemented**:

ID	Feature	ID	Feature
F04	Concept Extraction	F05	Question Answering Over Materials
F06	Personalized Study-Plan Generation	F07	Flashcard Generation
F08	Quiz Generation	F09	Automatic Quiz Grading
F10	Calendar	UC-11	Maintain Study Profile (supports F06)

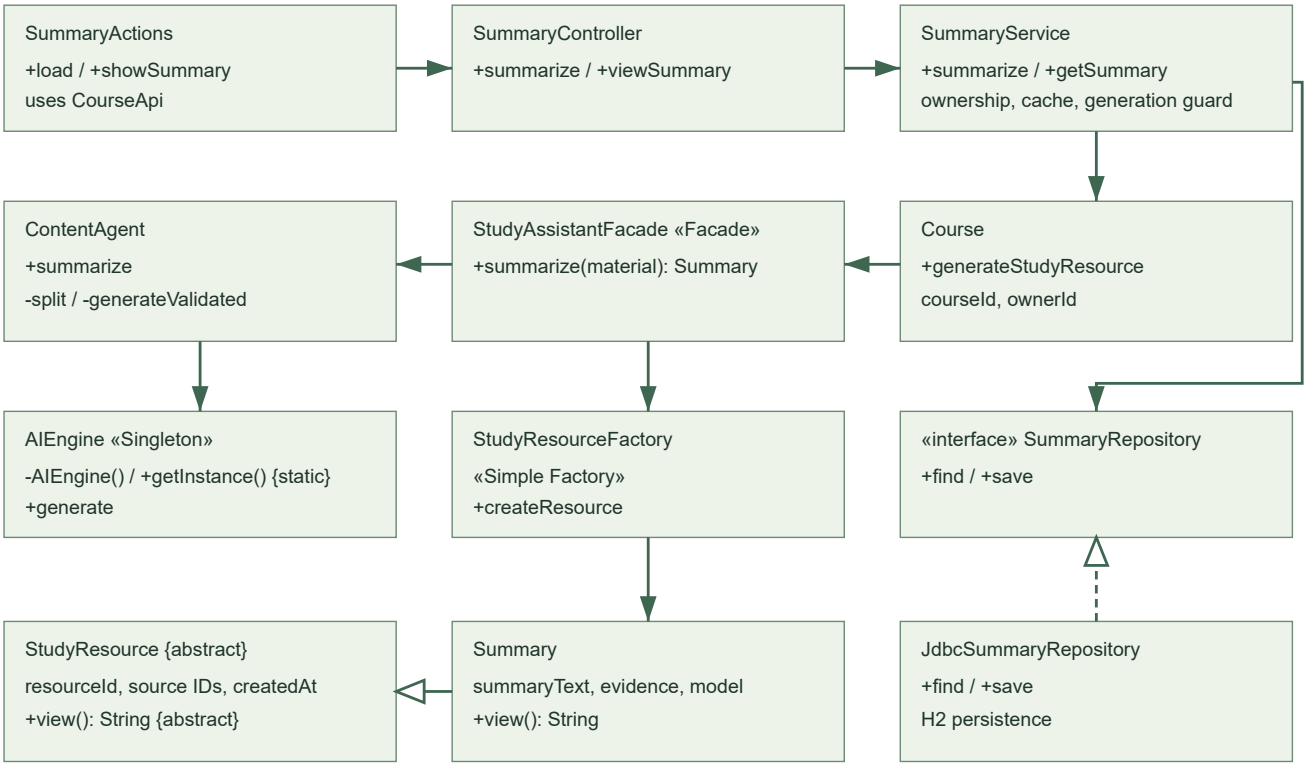
3. Design patterns and diagrams

Stage 1 pattern	Implementation and purpose
Facade	`StudyAssistantFacade.summarize` coordinates the agent and resource factory, hiding model orchestration and construction from course/UI callers.
Singleton	`AIEngine` has a private constructor and holder-based `getInstance`. Agents share one HTTP connection pool and concurrency limit; provider settings remain immutable per call.
Factory	`StudyResourceFactory.createResource` centralizes resource identity, timestamp, source association, and provenance. This simple factory currently constructs only `Summary`; other products remain pending.

Strategy and Observer remain pending. `CourseApiFactory` and Spring bean scopes are not counted as additional patterns.

F01 adds a Repository abstraction: `CourseService` depends on `CourseRepository`, implemented by `JdbcCourseRepository`. This keeps SQL separate from application logic; it does not replace a planned pattern.

F03 UML class relationships; F01/F02 diagrams are linked below:



Updated UML/sequence diagrams: `documentation/diagrams/stage2-f01.md`, `stage2-f02.md`, and `stage2-f03.md`. They show the implemented classes and pattern roles; original Stage 1 diagrams are preserved.

4. Changes and rationale

Stage 1 design	Implementation change	Reason and impact
UI → CourseController → User → Course (F01/UC-01/SD01).	Added CourseApi, CourseService, CourseRepository, JDBC, and H2.	Connects the separate apps and saves courses across restarts. The creation sequence is updated.
User owns a collection of courses.	User creates courses; repository queries retrieve them by owner UUID derived from username.	Supports persistent ownership. Changing the username changes the owner identity; account persistence is future work.

Authenticated student required; authentication mechanism unspecified.	One configured account, Spring Security, HTTP Basic, and CSRF protection.	Enforces the precondition. Credentials stay in ignored local configuration. Remote deployment requires HTTPS.
Four course fields; detailed limits unspecified.	Required fields, text limits, trimming, and ordered dates.	Rejects invalid courses consistently. Editing/deletion remain pending.
GUI and CLI required.	Java CLI calls the same REST API as the GUI.	Reuses validation and storage. Future agent features still need CLI support.
F02/UC-02/SD01: CourseController → Course.addMaterial → LearningMaterial; file rules unspecified.	Added course navigation, MaterialService, parser, repository, text previews, and browser downloads of originals. PDF/DOCX/TXT, size and extraction limits apply.	Students can read extracted text or save the complete document, preserving formatting and images. Durable BLOB/CLOB storage keeps originals and text consistent. No AI or OCR is claimed.
F03/UC-03/SD02: Facade → ContentAgent → shared AIEngine → resource Factory → Summary.	Agent returns validated draft data for factory construction; section planning, synthesis, indexed source evidence, cache/source hashes, and explicit regeneration were added.	Handles longer documents and malformed model output with bounded work. SummaryService adds ownership and persistent memory. These additions affect the F03 sequence; original pattern roles are retained.
Stage 1 proposed ChatGPT, LangChain/RAG, and shared AIEngine.	Direct Java HTTP orchestration; holder-based singleton accepts provider settings per call. Local Ollama is selected.	Keeps the existing Java apps and enables local inference. RAG for later question-answering remains pending; no full-system RAG implementation is claimed.

No original feature or planned pattern has been removed or replaced.

5. AI tools and collaboration

Development tool: OpenAI Codex, a GPT-6-based coding agent; exact runtime model ID unavailable. Used for F01–F03 code, tests, diagrams, and reporting. **Application LLM:** local Ollama with qwen2.5:1.5b; no hosted API billing.

Interaction: the student requested F01/F02, a shorter report, then F03 and local Ollama. Codex implemented and checked the course, material, and summary flows.

Tokens and cost: Codex totals remain unavailable; add platform usage/billing before submission. Successful summaries retain provider-reported token counts; -1 means unavailable. Local compute cost is not measured. Human review remains required.

6. Development and verification

Twenty-five backend tests and the production frontend build pass. Coverage includes F01/F02 regression, F03 ownership/CSRF, section planning, output repair, quotation checks,

cache/regeneration, timeouts, provider errors, and concurrent generation. Scripted-provider Chrome/CLI checks pass for summary progress, dialogs, saved results, regeneration, errors, mobile layout, and persistence. These fixtures test integration, not model quality. Real qwen2.5:1.5b verification also passed: CLI generation, browser viewing/regeneration, and a two-section document synthesized in three model calls. Source quotations were checked and the sample output reviewed. This verifies a working local integration, not general model accuracy.

Checks: `backend/mvnw.cmd clean verify`, frontend `mvnw.cmd package -Pproduction`, and `scripts/verify-f01.ps1`. Runtime checks use isolated test data and stop their own servers. A missing BCrypt encoding prefix was found and corrected during testing.

F02 runtime checks use `scripts/verify-f01.ps1 -IncludeF02`. Real-client checks caught a text-preview Accept-header mismatch and a parallel-upload CSRF session race. Correct response headers and serialized token/write requests fixed them.

Live testing exposed paraphrased quotations and missing JSON fields in the small model; constrained output and indexed evidence corrected these failures. F03 checks: `scripts/verify-f01.ps1 -IncludeF03` uses a test-only provider; `scripts/verify-f03-live.ps1` uses real Ollama and a safe lecture sample. No production fake-response fallback exists.

Toronto reporting period	Minimum code commits	Observed local evidence
October 20–26, 2026	5	No qualifying local commits found; remote evidence needs checking.
October 27–November 2, 2026	5	a0db921: starter; 42853d5: F01; 8ab568b: F02. F03 uncommitted; push status not verified.
November 3–9, 2026	5	Pending.

Local credentials and database files are ignored by Git. `backend/config.local.properties.example` provides placeholders; setup instructions are in the README. F01 requires no API key.

Before November 10: complete F04–F10, Strategy/Observer and additional resource products, full-system diagrams, videos, cost records, and commit evidence. Check additional baseline instructions; only the supplied Stage 1 report and Stage 2 instructions were reviewed.

7. Demonstration index

Feature	Video status	Demonstration scope
F01 Course Creation	Recording and accessible link/timestamp pending.	Identify F01; show GUI and CLI creation, input errors, dashboard output, and persistence. Explain that F01 is deterministic. Keep credentials hidden.

F02 Upload Learning Materials	Recording and link/timestamp pending.	Show course navigation, GUI/CLI upload, saved files, preview, original downloads, invalid/type/size errors, and persistence. Identify deterministic extraction and pending AI processing.
F03 Document Summarization	Recording/link pending.	Show real Ollama GUI/CLI generation, source quotations, saved summaries, regeneration, and error cases. Explain section planning and output validation.
F04–F10 and UC-11	Pending implementation.	Add a labeled video segment for each implemented feature.

Automated browser checks do not replace demonstration videos. Update this source and regenerate `stage2-report.pdf` as features are completed.